RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Ex1: Data Preprocessing and Cleaning

Aim: To clean and preprocess the Telco Customer Churn dataset by handling duplicates and missing values, encoding and scaling the features, and visualising the data.

Problem Statement:
A telecommunications company is experiencing customer churn (customers leaving the service). Before building a machine learning model to predict churn, the data science team must clean and preprocess the customer data to ensure high-quality input for analysis.

You are a Data Scientist working for the company. Your task is to preprocess the customer dataset by handling missing values, removing inconsistencies, transforming categorical features, scaling numerical attributes, and visualizing important relationships.

Learning Objective:
The objective of this experiment is to enable students to understand and perform the essential steps involved in data preprocessing. This experiment will provide a strong foundation for preparing high-quality data for subsequent machine learning and predictive analytics tasks.

Tasks to Perform:

1. Import the required Python libraries.
2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.
3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records
4. Clean the data
    a. Remove duplicate customer records.
    b. Replace missing values
        i. Numerical columns → Median
        ii. Categorical columns → Mode
    c. Verify that no missing values remain.
5. Encode categorical columns: Gender, Partner, Dependents, Contract, PaymentMethod, InternetService, Churn. Use appropriate encoding techniques.
6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.
7. Plot Distribution of MonthlyCharges, Distribution of Tenure.
8. Create a Count Plot showing Contract vs Churn.
9. Draw a Box Plot of MonthlyCharges grouped by Churn.
10. Display the Correlation Heatmap of all numerical features.

Solution:

```
# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```



RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

```
from sklearn.preprocessing import LabelEncoder, StandardScaler

# 2. Load dataset
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print(df.head(10))
print(df.tail(10))
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())

# 3. Explore dataset
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df.info()
print(df.describe())
print("Missing values:\n", df.isnull().sum())
print("Duplicate records:", df.duplicated().sum())

# 4. Clean data
df = df.drop_duplicates()
num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(exclude=np.number).columns.drop("customerID")
for c in num_cols:
    df[c] = df[c].fillna(df[c].median())      # numerical -> median
for c in cat_cols:
    df[c] = df[c].fillna(df[c].mode()[0])     # categorical -> mode
print("Missing values left:", df.isnull().sum().sum())
df_clean = df.copy()                          # keep readable labels for plots

# 5. Encoding
for c in ["gender", "Partner", "Dependents", "Churn"]:      # binary -> Label
    df[c] = LabelEncoder().fit_transform(df[c])
df = pd.get_dummies(df, columns=["Contract", "PaymentMethod",
                                 "InternetService"], dtype=int)  # One-Hot

# 6. Standardise numerical features
num = ["tenure", "MonthlyCharges", "TotalCharges"]
df[num] = StandardScaler().fit_transform(df[num])
print(df[num].describe().round(2))

# 7-8. Distributions
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df_clean["MonthlyCharges"], kde=True, ax=ax[0])
sns.histplot(df_clean["tenure"], kde=True, ax=ax[1])
ax[0].set_title("Distribution of MonthlyCharges")
ax[1].set_title("Distribution of Tenure")
plt.show()

# 9. Count plot: Contract vs Churn
sns.countplot(data=df_clean, x="Contract", hue="Churn")
plt.title("Contract vs Churn")
plt.show()

# 10. Box plot: MonthlyCharges by Churn
sns.boxplot(data=df_clean, x="Churn", y="MonthlyCharges")
plt.title("MonthlyCharges by Churn")
plt.show()

# 11. Correlation heatmap
corr = df[["SeniorCitizen", "tenure", "MonthlyCharges",
           "TotalCharges", "Churn"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

RASHIGA B (241801225)



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Result:

The Telco dataset (7,043 rows, 21 columns) was cleaned and preprocessed. There were no duplicates and the 11 missing TotalCharges values were filled with the median. The categorical columns were encoded, the numerical columns were standardised and all the plots were generated.

RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Ex2: Data Discovery and Preparation

Aim: To explore the Telco Customer Churn dataset, find outliers using the IQR method, study the factors affecting churn and prepare the data for modelling.

Learning Objective:
The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Question:
A telecommunications company has observed an increase in customer churn, leading to revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset, identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform:

1. Reuse Ex:1 Steps 1- 6, 10.
2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method.
3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms and Boxplots.
4. Detect outliers in numerical features using the Interquartile Range (IQR) method.
5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study the relationship between: Contract Type and Churn, Internet Service and Churn, Monthly Charges and Churn, Tenure and Churn.
6. Do the below analytical questions:
   - Which contract type has the highest customer churn?
   - Which payment method is most frequently used by customers who churn?
   - Do customers with shorter tenure tend to churn more often?
   - Which customer group has the highest monthly charges?
   - Which Internet Service category experiences the highest churn?
   - Is there any relationship between monthly charges and churn?
   - Which features appear to be the strongest indicators of customer churn?
7. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.
8. Save the final processed dataset as a new CSV file named CustomerChurn_Preprocessed.csv.

Solution:

```
# Reused from Ex1 (import, load, explore, clean)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```



RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

```
df = df.drop_duplicates()
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())

# Distribution of Churn, Gender, Contract, PaymentMethod
fig, ax = plt.subplots(1, 4, figsize=(20, 4))
for a, c in zip(ax, ["Churn", "gender", "Contract", "PaymentMethod"]):
    sns.countplot(data=df, x=c, ax=a)
    a.tick_params(axis="x", rotation=25)
plt.tight_layout()
plt.show()
for c in ["Churn", "gender", "Contract", "PaymentMethod"]:
    print((df[c].value_counts(normalize=True) * 100).round(1))

# Histograms and boxplots
num = ["MonthlyCharges", "TotalCharges", "tenure"]
fig, ax = plt.subplots(2, 3, figsize=(15, 7))
for i, c in enumerate(num):
    sns.histplot(df[c], kde=True, ax=ax[0, i])
    sns.boxplot(x=df[c], ax=ax[1, i])
plt.tight_layout()
plt.show()

# Outlier detection using IQR
for c in num:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    out = df[(df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)]
    print(f"{c}: {len(out)} outliers")

# Relationship with churn
fig, ax = plt.subplots(2, 2, figsize=(12, 9))
sns.countplot(data=df, x="Contract", hue="Churn", ax=ax[0, 0])
sns.countplot(data=df, x="InternetService", hue="Churn", ax=ax[0, 1])
sns.boxplot(data=df, x="Churn", y="MonthlyCharges", ax=ax[1, 0])
sns.boxplot(data=df, x="Churn", y="tenure", ax=ax[1, 1])
plt.tight_layout()
plt.show()

# Analytical questions
yes = lambda s: (s == "Yes").mean() * 100
print(df.groupby("Contract")["Churn"].apply(yes).round(1))              # Q1
print(df[df.Churn == "Yes"]["PaymentMethod"].value_counts())            # Q2
print(df.groupby("Churn")[["tenure", "MonthlyCharges"]].mean().round(1)) # Q3,Q6
print(df.groupby("InternetService")["MonthlyCharges"].mean().round(1))   # Q4
print(df.groupby("InternetService")["Churn"].apply(yes).round(1))        # Q5
tmp = df.drop(columns="customerID").copy()
for c in tmp.select_dtypes(exclude=np.number):
    tmp[c] = LabelEncoder().fit_transform(tmp[c])
print(tmp.corr()["Churn"].abs().sort_values(ascending=False).head(6).round(3))  # Q7

# Encode, scale, split and save
df = df.drop(columns="customerID")
for c in ["gender", "Partner", "Dependents", "Churn", "PhoneService",
          "PaperlessBilling"]:
    df[c] = LabelEncoder().fit_transform(df[c])
df = pd.get_dummies(df, dtype=int)
df[num] = StandardScaler().fit_transform(df[num])

X, y = df.drop(columns="Churn"), df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Train:", X_train.shape, "Test:", X_test.shape)
df.to_csv("CustomerChurn_Preprocessed.csv", index=False)
```

RASHIGA B (241801225)

RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Answers:

1. Month-to-month contracts have the highest churn (42.7%, against 11.3% for one year and 2.8% for two year).

2. Electronic check is the most used payment method among churned customers (1,071 of 1,869).

3. Yes. Churned customers have an average tenure of about 18 months, while other customers have about 37.6 months.

4. Fiber optic customers have the highest monthly charges (average about 91.5).

5. Fiber optic has the highest churn (41.9%, against 19.0% for DSL and 7.4% for no internet).

6. Yes. Churned customers pay more on average (74.4) than the others (61.3).

7. Contract (0.40), tenure (0.35), OnlineSecurity (0.29), TechSupport (0.28) and TotalCharges (0.20) have the highest correlation with churn.

No outliers were found in MonthlyCharges, TotalCharges or tenure. The overall churn rate is 26.5%.

Result:

The data was explored and the questions were answered. It was split into 5,634 training and 1,409 testing records and saved as CustomerChurn_Preprocessed.csv.

Ex3a: Model Planning and Building - Classification

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate basic machine learning models using real-world datasets.

Aim: To build Logistic Regression, Decision Tree and Random Forest models to predict customer churn and compare their performance.

Problem Statement:
A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After completing data preprocessing and exploratory analysis, the company now aims to develop predictive models that can identify customers who are likely to churn. Early prediction of customer churn will enable the company to implement targeted retention strategies, improve customer satisfaction, and reduce business losses.

As a Data Scientist, your task is to build and compare multiple classification models to predict whether a customer will churn. You will train different machine learning algorithms, evaluate their performance using appropriate classification metrics, and identify the most effective model for customer churn prediction. Finally, interpret the important factors influencing customer churn and communicate your findings through suitable visualizations and performance comparisons.

Tasks to Perform:

1. Use CustomerChurn_Preprocessed.csv file which you created in Ex2.
2. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.

Model Building

3. Train a Logistic Regression classifier and predict the customer churn on the test dataset.
4. Train a Decision Tree Classifier and predict the customer churn on the test dataset.
5. Train a Random Forest Classifier and predict the customer churn on the test dataset.

Model Evaluation

6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score.
7. Compare the performance of all three models using a single bar chart showing the evaluation metrics.
8. Identify the best-performing model based on the evaluation results.

Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

Solution:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

df = pd.read_csv("CustomerChurn_Preprocessed.csv")
X, y = df.drop(columns="Churn"), df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
}
results = {}
for name, m in models.items():
    pred = m.fit(X_train, y_train).predict(X_test)
    print(name)
    print(confusion_matrix(y_test, pred))
    results[name] = {
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1-Score": f1_score(y_test, pred)}
res = pd.DataFrame(results).T.round(3)
print(res)

res.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Model Comparison")
plt.ylabel("Score")
plt.show()
print("Best model (by F1):", res["F1-Score"].idxmax())

rf = models["Random Forest"]
print(pd.Series(rf.feature_importances_, X.columns)
        .sort_values(ascending=False).head(5).round(3))
```

Output:

Model	Accuracy	Precision	Recall	F1-Score	Confusion matrix
Logistic Regression	0.806	0.658	0.561	0.606	[[926, 109], [164, 210]]
Decision Tree	0.717	0.468	0.489	0.478	[[827, 208], [191, 183]]
Random Forest	0.783	0.610	0.503	0.551	[[915, 120], [186, 188]]

Logistic Regression performs best on all the metrics.

Recommendations:

1. Offer discounts to month-to-month customers to move them to one or two year contracts.
2. Give extra attention to new customers, since churn is higher in the early months.
3. Promote online security and tech support services, as customers without them churn more.

Result:

Logistic Regression gave the best result with an accuracy of 0.806 and an F1-score of 0.606, followed by Random Forest and Decision Tree.

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Ex3b: Model Planning and Building - Regression

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate regression models using a real-world dataset.

Aim: To build Linear, Polynomial and SVR models to predict sales from advertising spend and compare them using MAE, MSE, RMSE and R2 score.

Problem Statement:
A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels—TV, Radio, and Newspaper—and wants to understand how these advertising expenditures influence product sales. By developing predictive models, the company aims to estimate future sales based on advertising investments and identify the most suitable regression model for accurate sales prediction.

As a Data Scientist, your task is to build and compare multiple regression models using the Advertising dataset. Evaluate the performance of each model using appropriate regression metrics and communicate the findings through visualizations to support business decision-making.

Tasks to Perform:

1. Download the Advertising Dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.
3. Select: TV, Radio, Newspaper as Input Features (X), Sales as Target Variable (y).

Perform Exploratory Data Analysis:

4. Generate a Pair Plot for the selected features.
5. Plot a Correlation Heatmap.
6. Identify the feature most strongly correlated with Sales.

Model Planning and Model Building

7. Split the dataset into Training (80%) and Testing (20%) dataset.
8. Build a Linear Regression model and predict Sales for the test dataset.
9. Build a Polynomial Regression model (Degree = 2) and predict Sales for the test dataset.
10. Build a Support Vector Regression (SVR) model with the RBF kernel and predict Sales for the test dataset.

Model Evaluation

11. Evaluate all three models using: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 Score
12. Compare the performance of all models using a single grouped bar chart.
13. Plot Actual vs Predicted Sales for the best-performing model.

Model Interpretation

14. Identify the model with the lowest prediction error.
15. Suggest **three recommendations** to help the company optimize its advertising budget based on the model results.

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.svm import SVR
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             r2_score)

df = pd.read_csv("advertising.csv")
print(df.head())
print(df.tail())
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe())

X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]

# EDA
sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
print(df.corr()["Sales"].drop("Sales").sort_values(ascending=False))

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Models
lr = LinearRegression().fit(X_train, y_train)
pred_lr = lr.predict(X_test)

poly = PolynomialFeatures(degree=2)
pr = LinearRegression().fit(poly.fit_transform(X_train), y_train)
pred_pr = pr.predict(poly.transform(X_test))

sc = StandardScaler().fit(X_train)
svr = SVR(kernel="rbf").fit(sc.transform(X_train), y_train)
pred_svr = svr.predict(sc.transform(X_test))

# Evaluation
preds = {"Linear": pred_lr, "Polynomial": pred_pr, "SVR": pred_svr}
rows = {}
for n, p in preds.items():
    mse = mean_squared_error(y_test, p)
    rows[n] = {"MAE": mean_absolute_error(y_test, p), "MSE": mse,
               "RMSE": np.sqrt(mse), "R2": r2_score(y_test, p)}
res = pd.DataFrame(rows).T.round(3)
print(res)
res.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Regression Model Comparison")
plt.show()

best = res["RMSE"].idxmin()
print("Lowest error model:", best)
```

```
plt.scatter(y_test, preds[best])
plt.plot([y.min(), y.max()], [y.min(), y.max()], "r--")
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title(f"Actual vs Predicted ({best})")
plt.show()
```

Output:

The dataset has 200 rows with no missing or duplicate values. TV has the highest correlation with Sales (0.78), followed by Radio (0.58) and Newspaper (0.23).

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.461	3.174	1.782	0.899
Polynomial Regression	0.526	0.413	0.643	0.987
SVR (RBF)	1.162	2.862	1.692	0.909

Polynomial Regression has the lowest error, so the Actual vs Predicted plot is drawn for it.

Recommendations:

1. Spend the most on TV advertising, since it has the strongest effect on sales.
2. Use Radio along with TV, as the combination gives better sales.
3. Reduce the Newspaper budget because its effect on sales is very small.

Result:

Polynomial Regression gave the lowest error (RMSE 0.643) and the highest R2 score (0.987) among the three models.

Ex4: Information Retrieval using NLTK

Learning Objective:
The objective of this experiment is to enable students to understand the text preprocessing pipeline and implement a basic information retrieval system using TF-IDF vectorization, Cosine Similarity, NLTK.

Aim: To preprocess customer reviews using NLTK and build a retrieval system using TF-IDF and cosine similarity.

Problem Statement:
An e-commerce company receives thousands of customer reviews for its products every day. Customer support executives often need to retrieve reviews that are similar to a user's search query (e.g., "great product with fast shipping" or "poor quality") to quickly understand customer feedback and address common concerns.

As an NLP Engineer, your task is to preprocess customer review text, convert the reviews into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, and develop a simple information retrieval system that returns the most relevant customer reviews for a given search query using Cosine Similarity.

Tasks to Perform:

1. Use Amazon Fine Food Reviews Dataset from Kaggle and load it into a Pandas DataFrame.
2. Install and import the required Python libraries:
3. Import the required NLTK resources:

Clean the dataset

4. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing.

Text Preprocessing

5. Create a preprocessing function to perform the following operations on each review:
   Convert text to lowercase, Remove punctuation, numbers, and special characters, Tokenize the text into words, Remove English stopwords, Join the cleaned tokens into a single string.
6. Apply the preprocessing function to all reviews and store the cleaned reviews in a new column named Cleaned_Text.
7. Display the original and cleaned versions of any five reviews.
8. Convert the cleaned review text into TF-IDF feature vectors. Display: Shape of the TF-IDF matrix, Total number of unique terms (vocabulary size).

Information Retrieval

10. Implement a document retrieval function that: Accepts a user query, Applies the same preprocessing steps used for the reviews, Converts the query into a TF-IDF vector, Computes Cosine Similarity between the query and all review vectors, Retrieves the Top 5 most similar reviews, Display Similarity score, Original review, Cleaned review.

Query Evaluation

11. Test the retrieval system using the following sample queries: "great product with fast shipping", "disappointed", "excellent taste", "poor packaging"
12. Compare the retrieved reviews for different queries and comment on their relevance.

Solution:

```
# pip install nltk scikit-learn pandas
```

```
import re
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")

# Load and clean the dataset
df = pd.read_csv("Reviews.csv")
df = df[["Text"]].dropna().drop_duplicates().head(10000).reset_index(drop=True)
print(df.shape)

# Text preprocessing
stop_words = set(stopwords.words("english"))

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)          # remove punctuation/numbers
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return " ".join(tokens)

df["Cleaned_Text"] = df["Text"].apply(preprocess)
for i in range(5):
    print("Original:", df["Text"][i])
    print("Cleaned :", df["Cleaned_Text"][i], "\n")

# TF-IDF
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df["Cleaned_Text"])
print("TF-IDF matrix shape:", X.shape)
print("Vocabulary size:", len(tfidf.vocabulary_))

# Retrieval
def search(query, top_k=5):
    q_vec = tfidf.transform([preprocess(query)])
    scores = cosine_similarity(q_vec, X).ravel()
    for i in scores.argsort()[::-1][:top_k]:
        print(f"Score   : {scores[i]:.4f}")
        print("Original:", df["Text"][i])
        print("Cleaned :", df["Cleaned_Text"][i], "\n")

queries = ["great product with fast shipping", "disappointed",
           "excellent taste", "poor packaging"]
for q in queries:
    print("=" * 70, "\nQUERY:", q)
    search(q)
```

Observation:

Reviews that contain the words in the query get the highest similarity scores. Queries with more matching words return more relevant reviews. Reviews with similar meaning but different words (for example "bad" for "poor") are not retrieved because TF-IDF matches only the words.

Result:

The reviews were preprocessed, converted to TF-IDF vectors, and the top 5 similar reviews were retrieved for each query.

Ex5: Information Extraction using spaCy

Aim: To extract tokens, lemmas, POS tags and named entities from customer reviews using spaCy.

Learning Objective:
The objective of this experiment is to enable students to understand and implement information extraction techniques on unstructured customer review text using spaCy.

Problem Statement:
An e-commerce company receives thousands of customer reviews containing valuable information about products, brands, organizations, locations, quantities, and other entities. Manually identifying and organizing this information from large volumes of reviews is time-consuming and inefficient.

As an NLP Engineer, your task is to process customer reviews using the spaCy Natural Language Processing library and develop an information extraction system that identifies and extracts meaningful linguistic information and named entities from the reviews. The extracted information should be organized into a structured format that can be further used for customer feedback analysis.

Tasks to Perform:

1. Reuse the Amazon Fine Food Reviews Dataset used in the previous experiment and load it into a Pandas DataFrame. Install and import the required libraries.

Clean the dataset

2. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing.

Text Preprocessing and Part-of-speech (POS) tags

3. Apply the spaCy pipeline to the review text. For any five reviews, display: Original review, Tokens, Lemmas, Part-of-speech (POS) tags.
4. Identify and display different POS categories such as: Nouns, Verbs, Adjectives, Adverbs.
5. Count and display the frequency of the major POS categories present in the reviews.

Named Entity Recognition (NER)

6. Apply Named Entity Recognition (NER) using spaCy to identify entities in the reviews. Extract and display entities such as: PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT, where recognized by the selected spaCy model. Display the extracted information in a structured DataFrame.

Information Extraction

7. Create a function that accepts a customer review as input and returns: Tokens, Lemmas, POS tags, Named entities, Entity labels.
8. Test the function using at least three new customer reviews.
9. Display the extracted information in a structured format.

Solution:

```
# pip install spacy pandas
# python -m spacy download en_core_web_sm
import pandas as pd
import spacy
from collections import Counter
```

```
nlp = spacy.load("en_core_web_sm")

# Load and clean the dataset
df = pd.read_csv("Reviews.csv")
df = df[["Text"]].dropna().drop_duplicates().head(10000).reset_index(drop=True)

# Tokens, lemmas and POS tags for five reviews
for text in df["Text"][:5]:
    doc = nlp(text)
    print("Review:", text)
    print("Tokens:", [t.text for t in doc])
    print("Lemmas:", [t.lemma_ for t in doc])
    print("POS   :", [(t.text, t.pos_) for t in doc], "\n")

# POS categories for the first review
doc = nlp(df["Text"][0])
for name, tag in [("Nouns", "NOUN"), ("Verbs", "VERB"),
                  ("Adjectives", "ADJ"), ("Adverbs", "ADV")]:
    print(name, ":", [t.text for t in doc if t.pos_ == tag])

# POS frequency and NER over all reviews
docs = list(nlp.pipe(df["Text"], batch_size=200, disable=["parser"]))
pos_count = Counter(t.pos_ for d in docs for t in d
                    if not t.is_punct and not t.is_space)
print(pd.DataFrame(pos_count.most_common(), columns=["POS", "Count"]))

wanted = {"PERSON", "ORG", "DATE", "MONEY", "QUANTITY", "PRODUCT"}
rows = [{"Review_ID": i, "Entity": e.text, "Label": e.label_}
        for i, d in enumerate(docs) for e in d.ents if e.label_ in wanted]
ner_df = pd.DataFrame(rows)
print(ner_df.head(20))
print(ner_df["Label"].value_counts())

# Information extraction function
def extract_info(review):
    doc = nlp(review)
    return {
        "Tokens": [t.text for t in doc],
        "Lemmas": [t.lemma_ for t in doc],
        "POS": [(t.text, t.pos_) for t in doc],
        "Entities": [e.text for e in doc.ents],
        "Entity Labels": [(e.text, e.label_) for e in doc.ents],
    }

tests = [
    "I bought Nestle coffee for $12 in January and loved the taste.",
    "The 2 lb pack from Amazon arrived late, but John said it was fresh.",
    "Kellogg's cereal costs 5 dollars at Walmart and my kids enjoy it.",
]
for t in tests:
    info = extract_info(t)
    print("\nReview:", t)
    print(pd.DataFrame({k: [v] for k, v in info.items()}).T)
```

Result:

Tokens, lemmas, POS tags and named entities were extracted from the reviews using spaCy and displayed in a structured format.

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Ex6a: Exploratory Data Analysis

Aim: To perform exploratory data analysis on the Netflix titles dataset and find patterns in the content.

Learning Objective:
The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA)on a real-world Netflix titles dataset using Python.

Problem Statement:
Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. Netflix wants to understand the characteristics and growth of its content library and identify important patterns in its movies and TV shows.

As a Data Scientist, perform an Exploratory Data Analysis on the Netflix Titles dataset. Analyse the content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions. Use suitable statistical techniques and visualisations to identify patterns and generate meaningful insights from the data.

Tasks to Perform:

1. Download Netflix_titles dataset from kaggle and convert it to dataframe.
2. Explore the Dataset
3. **Content Distribution Analysis. (Plot graphs wherever needed)**
    a. How many Movies vs TV Shows are in the dataset?
    b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?
    c. Which countries have the most Netflix content?
4. **Time-Based Trend Analysis (Plot graphs wherever needed)**
    a. In which years were the most shows/movies released?
    b. In which years were titles added to Netflix?
    c. How has Netflix's content growth trend changed over time?
5. **Duration & Content Type Analysis (Plot graphs wherever needed)**
    a. What is the distribution of movie durations (in minutes)?
    b. How many TV shows exist per number of seasons?
6. **Genre & Categories Analysis (Plot graphs wherever needed)**
    a. What are the top 10 most common genres on Netflix?
    b. Which directors have the most titles on Netflix?
    c. Which actors appear most frequently across Netflix shows/movies?
7. **Text & Descriptions**
    a. What are the most common words in the description field? (word cloud)
8. In which months of the year does Netflix add the most content?
9. Which countries produce more movies vs TV shows**?**
10. What are the most common ratings for Movies vs TV Shows?
11. What are the oldest and newest titles available on Netflix?
12. Which genres frequently co-occur together**?** (Genre correlation heatmap)

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch: 2024-2028**

Solution:

```
# pip install wordcloud
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from sklearn.preprocessing import MultiLabelBinarizer

df = pd.read_csv("netflix_titles.csv")
print(df.head())
print(df.shape)
df.info()
print(df.isnull().sum())
print(df.describe(include="all").T)

# ---------- Content distribution ----------
print(df["type"].value_counts())                                    # Q1
df["type"].value_counts().plot.pie(autopct="%1.1f%%")
plt.show()

print(df["rating"].value_counts())                                  # Q2
sns.countplot(data=df, y="rating", order=df["rating"].value_counts().index)
plt.show()

country = df.dropna(subset=["country"]).assign(
    country=lambda d: d["country"].str.split(", ")).explode("country")
country["country"] = country["country"].str.strip()
top_c = country["country"].value_counts().head(10)                  # Q3
print(top_c)
top_c.plot.barh()
plt.show()

# ---------- Time based trends ----------
df["date_added"] = pd.to_datetime(df["date_added"].str.strip())
df["year_added"] = df["date_added"].dt.year
df["month_added"] = df["date_added"].dt.month_name()

print(df["release_year"].value_counts().head(10))                   # Q4
df["release_year"].value_counts().sort_index().plot(figsize=(10, 4))
plt.title("Titles by release year")
plt.show()

print(df["year_added"].value_counts().head(10))                     # Q5
df["year_added"].value_counts().sort_index().plot(kind="bar")
plt.title("Titles added by year")
plt.show()

growth = df.groupby(["year_added", "type"]).size().unstack(fill_value=0)
growth.plot(figsize=(10, 4), marker="o")                            # Q6
plt.title("Content growth over time")
plt.show()

# ---------- Duration ----------
movies = df[df["type"] == "Movie"].copy()
movies["minutes"] = movies["duration"].str.extract(r"(\d+)").astype(float)
sns.histplot(movies["minutes"].dropna(), bins=30, kde=True)         # Q7
plt.show()

tv = df[df["type"] == "TV Show"].copy()
tv["seasons"] = tv["duration"].str.extract(r"(\d+)").astype(float)
```

```
print(tv["seasons"].value_counts().sort_index())                    # Q8
sns.countplot(data=tv, x="seasons")
plt.show()

# ---------- Genres, directors, actors ----------
def top_items(col, n=10):
    return (df[col].dropna().str.split(", ").explode().str.strip()
            .value_counts().head(n))

genres = top_items("listed_in")                                     # Q9
print(genres)
genres.plot.barh()
plt.show()
directors = top_items("director")                                   # Q10
print(directors)
directors.plot.barh()
plt.show()
actors = top_items("cast")                                          # Q11
print(actors)
actors.plot.barh()
plt.show()

# ---------- Text & descriptions ----------
text = " ".join(df["description"].dropna())                         # Q12
wc = WordCloud(width=900, height=450, background_color="white",
               stopwords=None).generate(text)
plt.imshow(wc)
plt.axis("off")
plt.show()

month_order = ["January", "February", "March", "April", "May", "June", "July",
               "August", "September", "October", "November", "December"]
months = df["month_added"].value_counts().reindex(month_order)    # Q13
print(months)
months.plot(kind="bar")
plt.show()

top10 = top_c.index
ct = (country[country["country"].isin(top10)]
      .groupby(["country", "type"]).size().unstack(fill_value=0))  # Q14
print(ct)
ct.plot(kind="bar", figsize=(10, 5))
plt.show()

print(pd.crosstab(df["rating"], df["type"]))                        # Q15

print("Oldest:", df.loc[df["release_year"].idxmin(), ["title", "release_year"]].tolist())   # Q16
print("Newest:", df.loc[df["release_year"].idxmax(), ["title", "release_year"]].tolist())

g = df["listed_in"].dropna().str.split(", ")                        # Q17
mlb = MultiLabelBinarizer()
G = pd.DataFrame(mlb.fit_transform(g), columns=mlb.classes_)
top_g = G.sum().sort_values(ascending=False).head(15).index
plt.figure(figsize=(10, 8))
sns.heatmap(G[top_g].corr(), cmap="coolwarm", annot=False)
plt.title("Genre co-occurrence (correlation)")
plt.show()
```

Result:

Exploratory data analysis of the Netflix titles dataset was done and the questions were answered

using the outputs and plots.

Ex6b: Time Series Data Analysis and Forecast

Aim: To check the stationarity of the airline passenger series and forecast it using an ARIMA model.

Learning Objective:
The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement:
An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform:

1. Download airline-passenges dataset from https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv
2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run ADF on the differenced series and visualize the new dataframe.
8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
9. Split the dataset for training and testing on original series.
10. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
11. Perform forecasts on the test dataset.
12. Compute metrics MSE, MAE, RMSE on the predictions.
13. Plot train, test, and predictions in a single plot.

Solution:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

df = pd.read_csv("airline-passengers.csv", parse_dates=["Month"],
                 index_col="Month")
```

```
print(df.head())
print(df.shape)
df.info()
print(df.describe())
df.plot(figsize=(10, 4), title="Monthly Airline Passengers")
plt.show()

def adf_test(s, name):
    stat, p = adfuller(s.dropna())[:2]
    print(f"{name}: ADF = {stat:.4f}, p-value = {p:.4f} ->",
          "Stationary" if p < 0.05 else "Non-stationary")

adf_test(df["Passengers"], "Original")
d1 = df["Passengers"].diff()
adf_test(d1, "1st difference")
d2 = d1.diff()
adf_test(d2, "2nd difference")
d2.dropna().plot(figsize=(10, 4), title="Second-order differenced series")
plt.show()

# Train/test split on the original series (80/20)
n = int(len(df) * 0.8)
train, test = df["Passengers"][:n], df["Passengers"][n:]

model = ARIMA(train, order=(2, 2, 2)).fit()      # d = 2 from stationarity check
forecast = model.forecast(steps=len(test))

mse = mean_squared_error(test, forecast)
print("MSE :", round(mse, 2))
print("MAE :", round(mean_absolute_error(test, forecast), 2))
print("RMSE:", round(np.sqrt(mse), 2))

plt.figure(figsize=(10, 5))
plt.plot(train, label="Train")
plt.plot(test, label="Test")
plt.plot(forecast, label="Forecast", linestyle="--")
plt.legend()
plt.title("ARIMA(2,2,2) Forecast")
plt.show()
```

Output:

Series	ADF statistic	p-value	Result
Original	0.8154	0.9919	Non-stationary
1st difference	-2.8293	0.0542	Non-stationary
2nd difference	-16.3842	0.0000	Stationary

The data has 144 monthly values, split into 115 for training and 29 for testing. The series became stationary after the second difference, so d = 2 was used in ARIMA(2, 2, 2).

MSE = 18382.92, MAE = 120.27, RMSE = 135.58

Result:

The series became stationary after second-order differencing. The ARIMA(2,2,2) model gave an RMSE of 135.58 on the test data.

Ex7: Model Planning and Building - Clustering

Learning Objective:
The objective of this experiment is to enable students to understand and implement the K-Means clustering algorithm for customer segmentation.

Aim: To group mall customers into segments using K-Means clustering on Annual Income and Spending Score.

Problem Statement:
A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. By identifying customers with similar purchasing behavior, the company can improve customer satisfaction, optimize promotional offers, and increase overall sales.

Your task is to analyze the customer dataset, apply the K-Means Clustering algorithm to identify customer segments, visualize the clusters, and recommend suitable marketing strategies for each segment.

Tasks to Perform:

1. Use Mall Customer Segmentation Dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.
3. Select the important features for clustering: Annual Income (k$), Spending Score (1–100).
4. Standardize the selected features using StandardScaler for clustering.
5. Visualize the selected features using: Scatter Plot, Pair Plot.

Model Planning and Building

6. Determine the optimal number of clusters using the Elbow Method. Plot the Elbow Curve and identify the optimal value of K.
7. Train the K-Means Clustering model using the optimal number of clusters. Assign cluster labels to each customer, add as a new column in the dataset and save as a file.

Cluster Visualization

8. Visualize the customer clusters using a scatter plot: X-axis: Annual Income, Y-axis: Spending Score, Different colors for each cluster, Highlight cluster centroids, Display the coordinates of the cluster centroids.

Cluster Analysis

9. Calculate the number of customers in each cluster.
10. Compute the average Annual Income and Spending Score for each cluster.

Solution:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("Mall_Customers.csv")
print(df.head())
print(df.tail())
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe())

features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

plt.scatter(X.iloc[:, 0], X.iloc[:, 1])
plt.xlabel(features[0])
plt.ylabel(features[1])
plt.show()
sns.pairplot(X)
plt.show()

# Elbow method
wcss = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_scaled).inertia_
        for k in range(1, 11)]
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Curve")
plt.show()

# Train with optimal K = 5
km = KMeans(n_clusters=5, random_state=42, n_init=10).fit(X_scaled)
df["Cluster"] = km.labels_
df.to_csv("Mall_Customers_Clustered.csv", index=False)

# Visualise clusters with centroids
centroids = scaler.inverse_transform(km.cluster_centers_)
sns.scatterplot(data=df, x=features[0], y=features[1],
                hue="Cluster", palette="tab10")
plt.scatter(centroids[:, 0], centroids[:, 1], s=250, c="black",
            marker="X", label="Centroids")
plt.legend()
plt.show()
print("Centroid coordinates (Income, Spending):\n", centroids.round(2))

# Cluster analysis
print(df["Cluster"].value_counts().sort_index())
print(df.groupby("Cluster")[features].mean().round(2))
```

Output:

The dataset has 200 customers with no missing or duplicate values. The elbow curve bends at K = 5, so 5 clusters were used.

Cluster	Customers	Avg Income (k$)	Avg Spending Score	Segment
0	81	55.30	49.52	Average income, average spending
1	39	86.54	82.13	High income, high spending
2	22	25.73	79.36	Low income, high spending
3	35	88.20	17.11	High income, low spending
4	23	26.30	20.91	Low income, low spending

The cluster centroids are the average values given in the table.

Marketing strategies:

Cluster 1: loyalty programs and premium offers.
Cluster 3: personalised promotions to increase their spending.
Cluster 2: discount coupons and reward points.
Cluster 0: seasonal sales and combo offers.
Cluster 4: low-cost products and clearance offers.

Result:

The customers were divided into 5 clusters using K-Means (K = 5 from the elbow method) and the cluster labels were saved to a file.

Ex8: Interactive Dashboards with Tableau

Learning Objective:
The objective of this experiment is to enable students to understand and implement interactive data visualisation and dashboard design using Tableau.

Aim: To create an interactive Sales and Profit dashboard in Tableau using the Superstore dataset.

Problem Statement:
Create an Interactive Dashboard in Tableau using the Sample – Superstore dataset from kaggle that includes:

- Bar chart (Sales by Category or Region)
- Line chart (Sales trend over time)
- Scatter plot (Profit vs Sales by Sub-Category)
- Interactive filters and dashboard actions

Tasks to Perform:

1. Open Tableau and Load dataset.
2. Explore the Data
3. **Create Worksheet 1 – Bar Chart (Sales by Category)**
   a. Click the **Sheet 1** tab → rename it `Bar_Sales_by_Category`.
   b. Drag `Category` to **Columns**.
   c. Drag `SUM(Sales)` to **Rows**.
   d. Sort the bars in descending order.
   e. Drag `Region` to **Color** → this gives different colors for each region.
   f. Add **Labels**: Click **Label** → check "Show Mark Labels".
   g. (Optional) Add filter: Drag `Region` to **Filters** → select all → then right-click it → "Show Filter".
4. **Create Worksheet 2 – Line Chart (Sales Trend Over Time)**
   a. Click **New Worksheet** → rename it `Line_Sales_Trend`.
   b. Drag `Order Date` to **Columns**.
   c. Right-click `Order Date` → select **Month** (continuous).
   d. Drag `SUM(Sales)` to **Rows**.
   e. Drag `Region` to **Color** (to show separate lines per region).
   f. Format the axis (if needed) → Right-click axis → "Format" → Currency.
   g. Show filter for Region or Category (optional).
5. **Create Worksheet 3 – Scatter Plot (Profit vs Sales)**
   a. Click **New Worksheet** → rename it `Scatter_Profit_vs_Sales`.
   b. Drag `SUM(Sales)` to **Columns**.
   c. Drag `SUM(Profit)` to **Rows**.
   d. Drag `Sub-Category` to **Detail** (each point = one Sub-Category).
   e. Drag `Category` to **Color**.

f. Optionally:
    i. Drag `Segment` to **Shape** (to add another dimension).
    ii. Add **Reference Line**: Analytics Pane → drag "Average Line" to both axes → "Table".

6. **Build the Dashboard**
    a. Click **New Dashboard** (bottom tab) → rename it `Sales Dashboard`.
    b. On the left, in the **Sheets** pane, drag and drop your 3 worksheets:
        i. Line chart at top
        ii. Bar chart bottom-left
        iii. Scatter plot bottom-right
    c. Adjust size (e.g., 1200×800).
    d. Use **containers** (horizontal/vertical) for cleaner alignment.
    e. Add **Title** → "Sales and Profit Dashboard".
7. **Add Interactivity**

(a) Add Filters

1. In Dashboard view → click dropdown on top-right of each sheet → "Show Filter".
2. Show filters for fields like **Region**, **Category**, or **Year**.

(b) Add Actions

1. Menu → **Dashboard → Actions → Add Action → Filter…**
2. Example:
    - **Source Sheet**: Bar chart (Sales by Category)
    - **Target Sheets**: Line and Scatter plots
    - **Run action on**: Select
    - **Clearing the selection**: Show all values
3. Add another **Highlight Action** (optional):
    - So hovering over a Sub-Category highlights the same category in other charts.

8. **Publish or Export**
    a. **To Tableau Public**: *File → Save to Tableau Public → sign in → publish.*
    b. **To PDF/Image**: *File → Export → Image/PDF.*

Solution:

The bar chart, line chart and scatter plot were created in separate worksheets by following the steps above and combined in the dashboard "Sales and Profit Dashboard". Region and Category filters and a filter action from the bar chart to the other two charts were added.

Result:

An interactive dashboard with a bar chart, line chart, scatter plot, filters and actions was created in Tableau.